

CIS106 Final Exam Study Notes

1. How to Clone a GitHub Repository

Step 1: Copy the Repository URL

Go to the GitHub repository page and click the green **Code** button. Copy the HTTPS URL.

Example

```
https://github.com/example/repo.git
```

Step 2: Open the Terminal

Open your Linux terminal in Debian.

Example

```
CTRL + ALT + T
```

Step 3: Go to the Directory Where You Want the Repository

Use the `cd` command to move into the folder where you want the repository downloaded.

Example

```
cd ~/Downloads
```

Step 4: Clone the Repository

Run the `git clone` command with the repository URL.

Example

```
git clone https://github.com/example/repo.git
```

2. How to Use Git Commands

Common Git Commands

Check status

```
git status
```

Shows modified files and current branch.

Add files

```
git add .
```

Adds all changed files.

Commit changes

```
git commit -m "message"
```

Saves changes locally.

Push changes

```
git push
```

Uploads changes to GitHub.

Pull latest changes

```
git pull
```

Downloads newest changes from GitHub.

3. How to Write a Markdown File That Contains Images and Proper Formatting

Step 1: Create a Markdown File

Use the `touch` command to create a markdown file.

Example

```
touch notes.md
```

Step 2: Add Proper Headings and Text Formatting

Markdown uses symbols for formatting text and organizing notes.

Example

```
# Main Heading
```

```
## Smaller Heading
```

Even Smaller Heading

Bold Text

Italic Text

Step 3: Add Code Blocks

Triple backticks are used for commands and code examples.

Example

```
ls -la
```

Step 4: Add an Image

Images in markdown use the following format.

Example

```
![Screenshot](image.png)
```

Another example:

```
![Linux Screenshot](q1.png)
```

Step 5: Preview the Markdown File

Open the markdown preview in VS Code to make sure everything looks clean and formatted correctly.

Example

```
CTRL + SHIFT + V
```

4. How to Convert a Markdown File to PDF

Step 1: Install Pandoc

Pandoc is used to convert markdown files into other formats like PDF.

Example

```
sudo apt install pandoc -y
```

Step 2: Make Sure You Are in the Correct Directory

Use the `cd` command to go to the folder that contains the markdown file.

Example

```
cd ~/cis106/FinalExamStudyNotes
```

Step 3: Check if the Markdown File Exists

Use the `ls` command to verify the markdown file is there.

Example

```
ls
```

Example output:

```
finalNotes.md
```

Step 4: Convert the Markdown File to PDF

Use `pandoc` to convert the markdown file into a PDF.

Example

```
pandoc finalNotes.md -o finalNotes.pdf
```

Step 5: Verify the PDF Was Created

Use the `ls` command again to make sure the PDF file was created successfully.

Example

```
ls
```

Example output:

```
finalNotes.md
```

```
finalNotes.pdf
```

5. How to Compress (Zip) a Directory/Folder in Debian

Step 1: Go to the Directory That Contains the Folder

Use the `cd` command to move into the location where the folder exists.

Example

```
cd ~/cis106
```

Step 2: Check if the Folder Exists

Use the `ls` command to verify the folder is there.

Example

```
ls
```

Example output:

```
FinalExamStudyNotes
```

Step 3: Compress the Folder Using zip

Use the `zip -r` command to compress the folder and all files inside it.

Example

```
zip -r FinalExamStudyNotes.zip FinalExamStudyNotes
```

Step 4: Verify the Zip File Was Created

Use the `ls` command again to confirm the zip file exists.

Example

```
ls
```

Example output:

```
FinalExamStudyNotes
```

```
FinalExamStudyNotes.zip
```

6. What Are Absolute Paths and Relative Paths?

Absolute Path

An absolute path starts from the root directory `/` and shows the complete location of a file or folder.

Example

```
/home/student/Documents/file.txt
```

Creating a File Using an Absolute Path

Example

```
touch /home/student/test.txt
```

Relative Path

A relative path starts from the current working directory.

Example

```
Documents/file.txt
```

Creating a File Using a Relative Path

Example

```
touch Documents/test.txt
```

Check Current Directory

Use the `pwd` command to see your current location.

Example

```
pwd
```

Example output:

```
/home/student
```

7. How to Work with the Manual Pages (man Command)

Step 1: Open a Manual Page

Use the `man` command followed by the command name.

Example

```
man ls
```

Step 2: Read Through the Manual Page

Use the keyboard to move through the page.

Examples

`Space` → next page

b → previous page

Arrow keys → scroll line by line

Step 3: Search Inside the Manual Page

Type **/** followed by the word you want to search.

Example

/sort

Step 4: Quit the Manual Page

Press **q** to exit.

Example

q

8. How to Parse (Search) for Specific Words in the Manual Page

Step 1: Open a Manual Page

Example

man grep

Step 2: Search for a Word

Type **/** followed by the word.

Example

/ignore

Step 3: Go to the Next Match

Press **n** to move to the next result.

Example

n

Step 4: Quit the Manual Page

Press **q**.

Example

```
q
```

9. How to Redirect Output (>, >>, and |)

Using >

The > symbol redirects output and overwrites the file.

Example

```
ls > files.txt
```

Using >>

The >> symbol appends output to a file without deleting existing content.

Example

```
echo "hello" >> notes.txt
```

Using |

The pipe | sends output from one command into another command.

Example

```
ls | less
```

Another example:

```
cat file.txt | grep hello
```

10. How to Append the Output of a Command to a File

Step 1: Run a Command

Commands generate output.

Example


```
ls
```

Step 2: Use the Append Operator >>

The >> operator adds output to the end of a file.

Example

```
ls >> files.txt
```

Step 3: Verify the File Content

Use the cat command to view the file.

Example

```
cat files.txt
```

Step 4: Append More Output

You can continue adding more content to the same file.

Example

```
echo "new line" >> files.txt
```

11. How and When to Redirect the Output of a Command to Another (Pipes)

A pipe | takes the output from one command and sends it into another command. This is useful when the output is too long or when you want to filter information quickly.

Basic Syntax

```
command1 | command2
```

Example 1

```
ls | less
```

This lets you scroll through the output one page at a time instead of everything showing at once.

Example 2

```
cat file.txt | grep hello
```

This searches for the word `hello` inside the file.

Example 3

```
history | tail
```

This shows the most recent commands used in the terminal.

Example 4

```
ps aux | grep firefox
```

This searches for Firefox processes currently running.

12. How to Use echo and Output Redirection to Create a New File That Contains Some Text

Step 1: Use the echo Command

The `echo` command prints text into the terminal.

Example

```
echo "Hello World"
```

Step 2: Redirect the Output into a File

Use `>` to create a new file with text inside it.

Example

```
echo "Hello World" > file.txt
```

Step 3: Check the File Content

Use `cat` to display the content of the file.

Example

```
cat file.txt
```

Example output:

```
Hello World
```

Step 4: Add More Text Without Deleting the Old Content

Use `>>` to append text to the file.

Example

```
echo "Second Line" >> file.txt
```

13. How to Use Wildcards (For Copying and Moving Multiple Files at the Same Time)

Wildcards are symbols that help work with multiple files at once.

Using the `*` Wildcard

The `*` matches everything.

Example

```
ls *.txt
```

This displays all text files.

Copy Multiple Files

Example

```
cp *.txt backup/
```

This copies all `.txt` files into the backup folder.

Move Multiple Files

Example

```
mv *.png images/
```

This moves all PNG image files into the images folder.

Using the `?` Wildcard

The `?` matches only one character.

Example

```
ls file?.txt
```

This matches files like:

file1.txt

file2.txt

but not:

file21.txt

14. How to Use Brace Expansion (For Creating Entire Directory Structures in a Single Command)

Brace expansion lets you create multiple files or folders using one command instead of typing everything separately.

Create Multiple Directories

Example

```
mkdir project/{css,html,js}
```

This creates:

- css
- html
- js

inside the project folder.

Create Multiple Files

Example

```
touch file{1..5}.txt
```

This creates:

- file1.txt
 - file2.txt
 - file3.txt
 - file4.txt
 - file5.txt
-

Another Example

```
mkdir semester/{notes,projects,labs}
```

This creates multiple folders at once.

15. How to Create a Simple “Hello World” Shell Script

Step 1: Create the Script File

Example

```
touch hello.sh
```

Step 2: Add the Script Content

Example

```
#!/bin/bash  
  
echo "Hello World"
```

Step 3: Give the Script Permission to Run

Example

```
chmod +x hello.sh
```

Step 4: Run the Script

Example

```
./hello.sh
```

Example output:

```
Hello World
```

16. How to Use Variables in a Shell Script

Variables are used to store values that can be reused later in the script.

Create a Variable

Example

```
name="JAKE"
```

Display the Variable

Example

```
echo $name
```

Example output:

```
JAKE
```

Full Script Example

Example

```
#!/bin/bash
```

```
name="JAKE"
```

```
echo "Hello $name"
```

Run the Script

Example

```
./script.sh
```

Example output:

```
Hello JAKE
```

Another Example

Example

```
age=69
```

```
echo $age
```

Example output:

```
69
```

17 For each of the following commands, include a definition, syntax/formula/usage/, and 2 - 5 well-documented examples.

awk

awk is used for processing and displaying text line by line.

Usage

```
awk [options] '{command}' file
```

Example 1

```
awk '{print $1}' cars.csv
```

Prints the first column from the file.

Example 2

```
awk '{print $2}' students.txt
```

Prints the second column from the file.

cat

cat is used to display the content of a file.

Usage

```
cat [option] file
```

Example 1

```
cat file.txt
```

Displays the content of file.txt

Example 2

```
cat -n file.txt
```

Displays the file with line numbers.

cp

cp is used to copy files and directories from one location to another.

Usage

```
cp source destination
```

Example 1

```
cp file.txt Documents/
```

Copies file.txt into the Documents folder.

Example 2

```
cp -r projects backup/
```

Copies the entire projects directory into the backup folder.

cut

cut is used to extract specific sections from each line of a file.

Usage

```
cut [option] file
```

Example 1

```
cut -d ":" -f1 /etc/passwd
```

Displays the usernames from the passwd file.

Example 2

```
cut -d ":" -f1,7 /etc/passwd
```

Displays usernames and their login shell.

grep

grep is used to search for specific text inside files.

Usage

```
grep [option] search file
```

Example 1

```
grep dracula dracula.txt
```

Searches for the word dracula inside the file.

Example 2

```
grep -i dracula dracula.txt
```


Searches for dracula without caring about uppercase or lowercase letters.

head

head displays the first lines of a file.

Usage

```
head [option] file
```

Example 1

```
head dracula.txt
```

Displays the first 10 lines of the file.

Example 2

```
head -5 dracula.txt
```

Displays the first 5 lines of the file.

ls

ls is used to list files and directories.

Usage

```
ls [option] directory
```

Example 1

```
ls
```

Lists files and folders in the current directory.

Example 2

```
ls -la
```

Lists all files including hidden files.

man

man is used to open the manual pages for Linux commands.

Usage

`man command`

Example 1

`man ls`

Opens the manual page for the ls command.

Example 2

`man grep`

Opens the manual page for the grep command.

mkdir

mkdir is used to create one or multiple directories.

Usage

`mkdir directory_name`

Example 1

`mkdir projects`

Creates a directory called projects.

Example 2

`mkdir homework labs notes`

Creates multiple directories at once.

mv

mv is used to move or rename files and directories.

Usage

`mv source destination`

Example 1

`mv file.txt Documents/`

Moves file.txt into the Documents folder.

Example 2

```
mv oldname.txt newname.txt
```

Renames the file from oldname.txt to newname.txt.

tail

tail displays the last lines of a file.

Usage

```
tail [option] file
```

Example 1

```
tail dracula.txt
```

Displays the last 10 lines of the file.

Example 2

```
tail -5 dracula.txt
```

Displays the last 5 lines of the file.

tac

tac displays the content of a file in reverse order.

Usage

```
tac [option] file
```

Example 1

```
tac helloWorld.py
```

Displays the file content from bottom to top.

Example 2

```
tac file1.txt file2.txt
```

Displays multiple files in reverse order.

tr

tr is used to translate or replace characters from standard output.

Usage

```
command | tr [option] set1 set2
```

Example 1

```
echo "hello" | tr a-z A-Z
```

Converts lowercase letters into uppercase letters.

Example 2

```
cat file.txt | tr "." ","
```

Replaces periods with commas.

tree

tree is used to display files and directories in a tree structure.

Usage

```
tree [directory]
```

Example 1

```
tree
```

Displays the current directory structure.

Example 2

```
tree Documents
```

Displays the structure of the Documents directory.

touch

touch is used to create files.

Usage

```
touch file_name
```

Example 1

```
touch list.txt
```

Creates a file called list.txt

Example 2

```
touch file1.txt file2.txt
```

Creates multiple files at once.